МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №7.

Выполнил: студент группы ___БВТ2402__
Скопа Михаил Вячеславович

Москва, 2025

Цель работы: познакомиться с многопоточностью.

Задание 1: Реализация многопоточнойпрограммы для вычисления сум мы элементов массива. Создать пул потоков с помощью класса Executor Service и разделить массив на равные части, каждую из которых будет обрабатывать отдельный поток. После завершения работы всех потоков результаты будут складываться в главном потоке.

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.concurrent.*;

public class ParallelArraySum {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        int arraySize = 1_000_000;
        int numThreads = 4;

        int[] array = new int[arraySize];
        Random random = new Random();

        for (int i = 0; i < array.length; i++) {
            array[i] = random.nextInt(bound: 10);
        }

        ExecutorService executor = Executors.newFixedThreadPool(numThreads);
        List<Future<Long>> futures = new ArrayList<>();

        int chunkSize = arraySize / numThreads;

        for (int i = 0; i < numThreads; i++) {
            final int start = i * chunkSize;
            final int end;
            if (i == numThreads - 1){
                end = arraySize;
            } else { end = start + chunkSize;}

            Callable<Long> task = () -> {
                long sum = 0;
                for (int j = start; j < end; j++) {
                    sum += array[j];
                }
                return sum;
            };

            futures.add(executor.submit(task));
        }

        long totalSum = 0;
        for (Future<Long> future : futures) {
            totalSum += future.get();
        }

        executor.shutdown();

        System.out.println("Сумма элементов массива: " + totalSum);
    }
```

Реализовал массив со случайными значениями. Далее создаю пулл потоков по заранее заданному значению. Разделяю весь массив на равные части по количеству потоков. Создаю счетчик, который будет указывать потоку какую часть массива исследовать. Задаю потоку задачу, добавляю

значение в массив ответов. Далее суммирую ответы всех потоков и получаю ответ.

Задание 2. Реализация многопоточной программы для поиска наибольшего элемента в матрице. Создать пул потоков с помощью класса Executor Service и разделить матрицу на равные части, каждую из которых будет обрабатывать отдельный поток. После завершения работы всех потоков результаты будут сравниваться в главном потоке для нахождения наибольшего элемента.

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.concurrent.*;


public class Matrix {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        int number_String = 10;
        int number_column = 10;
        int numThreads = number_String;

        Random random = new Random();

        int[][] matrix = new int[number_String][number_column];
        for (int i = 0; i < number_String; i++){
            for (int j = 0; j < number_column; j++){
                matrix[i][j] = random.nextInt(-100000, bound: 100000);
            }
        }
        for (int i = 0; i < matrix.length; i++){
            System.err.println(Arrays.toString(matrix[i]));
        }

        ExecutorService executor = Executors.newFixedThreadPool(numThreads);
        List<Future<Long>> futures = new ArrayList<>();

        for (int i = 0; i < number_String; i++){
            final int n = i;
            Callable <Long> task = ()->{
                long maximal = -1_000_000;
                for (int j = 0; j < number_column; j++){
                    if (matrix[n][j] > maximal){
                        maximal = matrix[n][j];
                    }
                }
                return maximal;
            };
            futures.add(executor.submit(task));
        }
        long TotalMaximal = -1_000_000;
        for (Future<Long> future : futures) {
            if (future.get() > TotalMaximal){
                TotalMaximal = future.get();
            }

        }

        executor.shutdown();

        System.out.println("Максимальный элемент матрицы:  " + TotalMaximal);

    }
}
```

Создал матрицу с изменяемыми параметрами. Заполнил ее случайными значениями. Создал пулл потоков и массив ответов потоков. Далее задал потокам задачи по вычислению наибольшего значения в строке матрицы. После сравнил ответы всех потоков и вывел окончательный ответ.

Задание 3. У вас есть склад с товарами, которые нужно перенести на другой склад. У каждого товара есть свой вес. На складе ра ботают 3 грузчика. Грузчики могут переносить товары одно временно, но суммарный вес товаров, переносимый ими за одну итерацию, не может превышать 150 кг. Как только грузчики со берут 150 кг товаров, они отправятся на другой склад и начнут разгружать товары. Напишите программу на Java, используя многопоточность, которая реализует данную ситуацию. Используйте CountDownLatch для синхронизации начала и завершения переноса товаров.

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;

class Warehouse {
    private List<Integer> goods;
    public int currentIndex = 0;

    public Warehouse(List<Integer> goods) {
        this.goods = goods;
    }

    public synchronized Integer getNextGood() {
        if (currentIndex < goods.size()) {
            return goods.get(currentIndex++);
        }
        return null;
    }

    public synchronized boolean hasGoods() {
        return currentIndex < goods.size();
    }
}
```

Создал класс Склад, который имеет задаваемый список с весами каждого товара, конструктор для этого списка, синхронизированные методы проверки наличия товаров на складе.

```java
class Worker implements Runnable {
    private String name;
    private Warehouse warehouse;
    private CountDownLatch startLatch;
    private CountDownLatch doneLatch;
    private static final int MAX_WEIGHT = 150;

    public Worker(String name, Warehouse warehouse, CountDownLatch startLatch, CountDownLatch doneLatch) {
        this.name = name;
        this.warehouse = warehouse;
        this.startLatch = startLatch;
        this.doneLatch = doneLatch;
    }

    @Override
    public void run() {
        try {
            startLatch.await();
            while (true) {
                List<Integer> load = new ArrayList<>();
                int totalWeight = 0;

                while (warehouse.hasGoods()) {
                    Integer weight = warehouse.getNextGood();
                    if (weight == null) break;

                    if (totalWeight + weight <= MAX_WEIGHT) {
                        load.add(weight);
                        totalWeight += weight;
                    } else {
                        synchronized (warehouse) {
                            warehouse.currentIndex = warehouse.currentIndex - 1;
                        }
                        break;
                    }
                }

                if (load.isEmpty()) {
                    break;
                }

                System.out.println(name + " переносит товары: " + load + " | Общий вес: " + totalWeight + " кг");

                Thread.sleep(millis: 1000);

                System.out.println(name + " разгрузил товары: " + load);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            doneLatch.countDown();
        }
    }
}
```

Создал класс погрузчиков с помощью интерфейса Runnable. Класс имеет конструктор, и метод run, который выполняет перевозку товаров.

```java
public class WarehouseTransfer {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws InterruptedException {
        List<Integer> goods = List.of(50, 70, 20, 30, 90, 60, 40, 10, 80, 20);

        Warehouse warehouse = new Warehouse(new ArrayList<>(goods));

        CountDownLatch startLatch = new CountDownLatch(count: 1);
        CountDownLatch doneLatch = new CountDownLatch(count: 3);

        Thread worker1 = new Thread(new Worker(name: "Грузчик 1", warehouse, startLatch, doneLatch));
        Thread worker2 = new Thread(new Worker(name: "Грузчик 2", warehouse, startLatch, doneLatch));
        Thread worker3 = new Thread(new Worker(name: "Грузчик 3", warehouse, startLatch, doneLatch));

        worker1.start();
        worker2.start();
        worker3.start();

        System.out.println(x: "Все грузчики готовы. Начинаем перенос товаров!");
        startLatch.countDown();

        doneLatch.await();
        System.out.println(x: "Все товары перенесены на другой склад!");
    }
}
```

Демонстрационный метод. Отображает возможности программы.

Вывод: познакомился с многопоточностью в языке программирования Java. Написал программу, использующую несколько потоков одновременно.

https://github.com/Mirphon/ITIP/tree/main/Lab7